

MÉTODO DECIDIR / DTIC IMIP, IN COMPANY

Da ideação à *aplicação publicada*

Doze passos entre uma dor real do backlog e uma URL que outra pessoa consegue abrir. Ninguém escreve código à mão.

12 passos

PRD e SDD

Claude Code

Supabase

Docker

12

PASSOS

A PERGUNTA QUE ABRE A IMERSÃO

Quanto tempo, hoje, entre a *ideia* e o no ar?

Na DTIC a resposta costuma ser medida em meses, e boa parte desse tempo não é construção: é espera, retrabalho e especificação que nunca foi escrita. A tarde de hoje ataca exatamente esse intervalo.

A stack da sala, uma só para todos

A restrição é deliberada. A equipe é capaz de mais, e é por isso que a stack é única: ela protege o marco da publicação. Desvio é decisão da dupla, com o risco de tempo assumido e registrado.

Next.js

App Router e TypeScript. Uma base só para telas e servidor, com build que empacota direto em container.

Supabase

Postgres gerenciado, com API pronta e controle de acesso por linha. Resolve a camada de dados em minutos, sem servidor para cuidar.

Docker e Traefik

A aplicação empacotada roda igual em qualquer máquina, e sobe atrás do proxy reverso com HTTPS automático.

A regra que atravessa o dia. Todo código do projeto tem origem em algo que você escreveu antes, em português, em um documento.

A régua de segurança

Não negociável, exposta agora e cobrada o dia inteiro. A pergunta que antecede qualquer colagem: eu mostraria isso numa auditoria?

Nenhum dado de paciente

Nem real, nem anonimizado por cima. A tarde inteira roda com dado sintético gerado pela própria IA.

Nenhum segredo no prompt

Senha, token, string de conexão, endereço de servidor e topologia ficam fora do prompt e do repositório.

Repositório privado, na organização

Nada em conta pessoal, nada em repositório público.

Nenhum dado de colaborador

Nome, matrícula, CPF e contato entram como fictícios.

Nenhuma integração com produção

O MVP de hoje é autônomo. Integração é decisão de arquitetura, não de oficina.

O humano assina

A IA propõe. A revisão e a responsabilidade continuam da pessoa e da diretoria.

O caminho inteiro, em uma tela

CLIQUE NO PASSO OU DIGITE O NÚMERO E ENTER

Cada passo produz um artefato, e o artefato de um passo é o insumo do seguinte.
Quem pula um passo paga no passo seguinte.

01 Os sete pilares o método	02 Desenvolvimento com IA o papel novo	03 Gerar o PRD docs/PRD.md	04 Gerar o SDD docs/SDD.md
05 Ideação com o Claude docs/DESIGN.md	06 Claude Code na IDE CLAUDE.md	07 Estrutura de pastas projeto versionado	08 Banco no Supabase schema.sql
09 Prompt de construção app rodando	10 Uso do GitHub repositório privado	11 Deploy com Docker container no ar	12 Projeto publicado URL com HTTPS



O método

Sete pilares aplicados a construir software. Cada um responde a uma pergunta e fecha com um artefato na mão.

7 pilares

Diagnóstico ao Resultado

Os sete pilares, aplicados a construir software

O mesmo método da formação base, lido na chave de quem vai entregar uma aplicação até o fim da tarde.

D Diagnóstico A dor real, recorrente, escrita em uma frase. PASSO 3	E Estratégia Escopo travado, com o que fica de fora explicitado. PASSO 3	C Comando O pedido que produz PRD, SDD e código útil. PASSOS 4 E 5	I Integração IDE, repositório, banco e ambiente, no lugar certo. PASSOS 6 A 8
D Delegação A IA escreve. Você conduz, verifica e commita. PASSO 9	I Inteligência Estressar o resultado antes que o usuário estresse. PASSO 11	R Resultado URL no ar, README e próxima decisão com data. PASSO 12	

D

Diagnóstico

A pergunta. Qual dor real, recorrente e minha vale virar aplicação?

Pilar 1 de 7

Especificação

O QUE A DUPLA FAZ

- 1 Descrever a rotina que hoje consome tempo, sem embelezar.
- 2 Separar irritação pontual de dor recorrente. Só dor recorrente vira software.
- 3 Escrever a dor em uma frase: hoje **[quem]** precisa **[fazer o quê]** e faz isso **[como]**, o que custa **[tempo, erro, retrabalho]**.
- 4 Confirmar com quem sofre a dor. Se a pessoa não reconhece a frase, a frase está errada.

ONDE ENTRA HOJE

Insumo do PRD, no passo 3

ENTREGÁVEL

A dor em uma frase

ARMADILHA

Propor o sistema grande que a instituição persegue há anos. O MVP correto é a menor fatia útil dele.



Estratégia

A pergunta. O que cabe no tempo que eu tenho, e o que fica de fora?

O QUE A DUPLA FAZ

- 1 Definir a única jornada principal, aquela que justifica a aplicação existir.
- 2 Escrever o não escopo. Escopo sem lista de exclusões não é escopo.
- 3 Fixar o critério de pronto: está pronta quando **[alguém]** consegue **[fazer o quê]** sem ajuda.
- 4 Adotar a stack padrão da sala e assumir o custo de qualquer desvio.

ONDE ENTRA HOJE

Escopo e seção Versão 2 do PRD

ENTREGÁVEL

Escopo travado, com exclusões explícitas

ARMADILHA

Trocar de ideia depois do protótipo validado. Dali em diante só muda a execução.



Comando

A pergunta. Como eu peço para receber exatamente o que me serve?

O QUE A DUPLA FAZ

- 1 Escrever o pedido em quatro partes: contexto, tarefa, formato e critério.
- 2 Mandar a IA perguntar antes de responder, uma pergunta por vez.
- 3 Criticar a resposta em vez de recomeçar do zero.
- 4 Guardar o comando que funcionou. Comando bom é ativo reutilizável.

ONDE ENTRA HOJE

Produz o PRD, o SDD e o prompt de construção

ENTREGÁVEL

Três comandos escritos e testados

ARMADILHA

Pedir tudo de uma vez. Pedido grande devolve resposta plausível e genérica.



Integração

A pergunta. Onde esse trabalho vive, para não morrer no meu notebook?

O QUE A DUPLA FAZ

- 1 Montar o ambiente: IDE, terminal e Claude Code autenticado.
- 2 Criar o repositório antes da primeira linha de código.
- 3 Definir onde ficam os dados e onde ficam os segredos, que nunca é o mesmo lugar.
- 4 Escrever o `CLAUDE.md`, que é o contrato entre você e a ferramenta.

ONDE ENTRA HOJE

Passos 6, 7 e 8

ENTREGÁVEL

Ambiente e repositório prontos

ARMADILHA

Começar a construir antes de ter git. Sem commit não existe volta.

D

Delegação

A pergunta. O que eu paro de fazer à mão a partir de hoje?

O QUE A DUPLA FAZ

- 1 Delegar a escrita do código, mantendo a decisão de arquitetura consigo.
- 2 Trabalhar em fatias, com critério de pronto explícito em cada uma.
- 3 Ver funcionando no navegador. Não acreditar no relato da IA.
- 4 Commitar assim que funcionar.

ONDE ENTRA HOJE

Passo 9, a construção

ENTREGÁVEL

A jornada principal funcionando localmente

ARMADILHA

Repetir o mesmo comando quando o erro persiste. Duas tentativas e muda de estratégia.



Inteligência

A pergunta. O que pode dar errado antes de alguém usar isso?

O QUE A DUPLA FAZ

- 1 Pedir à IA que ataque o próprio trabalho: onde quebra, o que falta, o que está inseguro.
- 2 Cobrir os quatro estados que a IA sempre esquece: vazio, carregando, erro e sucesso.
- 3 Revisar acessos ao banco e exposição de chaves.
- 4 Testar em tela pequena e com dado errado de propósito.

ONDE ENTRA HOJE

Revisão antes do deploy, passo 11

ENTREGÁVEL

Lista de riscos resolvidos e riscos aceitos

ARMADILHA

Confundir "não deu erro no meu computador" com "está correto".



Resultado

A pergunta. O que muda na segunda-feira?

Pilar 7 de 7

Publicação

O QUE A DUPLA FAZ

- 1 Publicar em URL acessível a outra pessoa.
- 2 Escrever o README: o que é, para quem, como roda, onde está e o que ainda não faz.
- 3 Definir dono técnico, dono da dor, situação e próxima decisão.
- 4 Medir: quantas vezes a aplicação foi usada na primeira semana.

ONDE ENTRA HOJE

Passo 12, a publicação

ENTREGÁVEL

URL, README e ficha de sustentação

ARMADILHA

Terminar a aplicação e não entregar a ninguém. Software sem usuário é exercício.



Especificação

Os três documentos que a IA lê antes de escrever a primeira linha. É aqui que a tarde é ganha ou perdida.

PRD

SDD

DESIGN

O que muda quando a IA escreve o código

O gargalo deixou de ser a digitação e passou a ser a clareza do pedido. Numa equipe que já sabe construir, é aqui que está o ganho.

O papel novo

Você deixa de ser quem digita e passa a ser quem especifica, decide e aprova. A IA é rápida, obediente e sem juízo próprio sobre o seu negócio. O juízo é seu.

A memória é externa

A IA esquece. O projeto não pode esquecer. PRD, SDD, DESIGN e CLAUDE.md são a memória do projeto em arquivo, dentro do repositório.

O ciclo

Pedir uma coisa só, ver funcionando, corrigir apontando comportamento, commitar. Ciclos curtos entregam; ciclos longos travam.

Desenvolvimento guiado por especificação. O PRD diz o que construir, o SDD diz como, e o prompt de construção só amarra os dois. Sem os documentos, a IA preenche as lacunas com suposição, e suposição de máquina custa retrabalho de gente.

O quadro que fica na parede

Nada aqui elimina engenharia. O que muda é onde o seu tempo é gasto.

Antes

Escrever código

Pesquisar sintaxe

Testar no fim

Documentar depois

Erro descoberto pelo usuário

Agora

Escrever especificação

Revisar proposta

Testar a cada fatia

Documentar antes, e o documento vira o código

Erro provocado por você, antes

Regra anti-loop. Dois erros iguais seguidos significam contexto ruim. Limpe o contexto, descreva o problema do zero ou volte ao último commit bom. Insistir no mesmo comando é o maior consumidor de tempo da tarde.

PRD, o documento do que

Product Requirements Document. Curto, em português, escrito com a IA e podado por você. É o único documento que o dono da dor precisa entender inteiro.

O QUE O PRD CONTÉM, E SÓ ISSO

- 1

O problema, na frase do Diagnóstico.
- 2

O usuário e o momento de uso.
- 3

A jornada principal, em passos numerados.
- 4

As telas, no máximo quatro.
- 5

Os dados guardados, com campos e tipos.
- 6

As regras de negócio que não são óbvias.
- 7

O que fica fora, na seção Versão 2.
- 8

O critério de pronto.

ENTREGÁVEL

`docs/PRD.md`, validado por quem sofre a dor.

SINAL DE SUCESSO

Você lê o PRD em voz alta em três minutos e outra pessoa entende o que será construído.

ARMADILHA

Aceitar o primeiro retorno. Ele vem maior do que cabe, porque a IA otimiza para parecer completa.

Os dois prompts do PRD

O primeiro produz o documento. O segundo é o que salva o dia, e quase ninguém usa.

PROMPT DO PRD

Você é meu parceiro de produto. Vou construir um MVP com IA em um dia, com Next.js e Supabase. Todos os dados serão sintéticos.

Dor: [colar a frase do Diagnóstico].

Antes de escrever, me faça as cinco perguntas cujas respostas mais mudam o resultado, uma de cada vez.

Depois escreva um PRD curto com: problema, usuário e momento de uso, a única jornada principal em passos, as telas (máximo quatro), os dados guardados com campos e tipos, as regras de negócio, a seção Versão 2 e o critério de pronto. Em português do Brasil, direto, sem jargão de consultoria.

PROMPT DA PODA

Corte este PRD pela metade. Mantenha apenas o indispensável para a jornada principal funcionar de ponta a ponta.

Tudo que for desejável mas não indispensável vai para a Versão 2. Devolva o PRD podado inteiro.

Na prática. A poda é o momento em que a dupla decide o que não vai construir hoje. Sem ela, o escopo cresce até o marco da publicação ser perdido.

SDD, o documento do como

Software Design Document. É o PRD traduzido em decisões técnicas, e é ele que impede a IA de inventar arquitetura no meio da construção.

PRD
Linguagem do negócio
O que o usuário faz
Validado pelo dono da dor
Muda quando o escopo muda

SDD
Linguagem técnica
Como o sistema faz
Validado por quem vai sustentar
Muda quando a decisão técnica muda

O que o SDD contém. Stack com justificativa de uma linha por item, arquitetura em blocos, modelo de dados em SQL, regras de acesso por tabela, telas e rotas, variáveis de ambiente, tratamento de erro, estratégia de deploy e o plano de entrega em fatias, na ordem de construção.

O prompt do SDD

As restrições entram no prompt. Sem elas, a IA propõe arquitetura grande, e arquitetura grande não cabe em uma tarde.

PROMPT DO SDD

Leia o PRD abaixo e escreva o SDD correspondente.

Restrições: aplicação web em Next.js com App Router e TypeScript, banco Postgres gerenciado no Supabase, deploy em container Docker atrás do Traefik com HTTPS. Sem microserviços, sem fila, sem cache distribuído, sem autenticação nesta versão.

Entregue: stack com justificativa de uma linha por item; arquitetura em blocos; modelo de dados com tabelas, colunas, tipos e relações, em SQL de criação; regras de acesso por tabela; telas e rotas; variáveis de ambiente; tratamento de erro; estratégia de deploy; e o plano de entrega dividido em fatias pequenas, na ordem de construção, cada uma com critério de pronto.

Onde faltar informação, liste a pergunta em vez de supor.

O QUE SAI DESTE PROMPT

Entregável

`docs/SDD.md`, com o SQL do schema já dentro.

Sinal de sucesso

O plano tem de cinco a oito fatias e você explica cada uma em uma frase.

Armadilha

Deixar passar fila, cache ou serviço separado no MVP. Se aparecer, corte.

Ver antes de construir

Um protótipo clicável custa vinte minutos e evita horas de código escrito para a tela errada. Quem valida é o dono da dor, não a dupla.

O QUE SE FAZ NESTE BLOCO

- 1 Pedir ao Claude um artefato: protótipo navegável, arquivo único, dados sintéticos realistas.
- 2 Definir a direção de arte antes: sóbria e institucional, uma única cor de destaque para a ação principal.
- 3 Três rodadas de crítica, uma coisa por rodada.
- 4 Teste dos cinco segundos: mostrar para alguém e perguntar o que a aplicação faz.
- 5 Congelar as decisões visuais em `docs/DESIGN.md`, que vira insumo do código.

ENTREGÁVEL

`prototipo.html` e `docs/DESIGN.md`.

SINAL DE SUCESSO

O dono da dor diz "é isso" ou aponta exatamente o que está errado.

ARMADILHA

Partir para o código com a interface na imaginação. Refazer tela custa mais que refazer protótipo.

Os prompts da ideação visual

Três momentos: gerar, criticar e congelar. A crítica é sempre uma coisa por vez.

GERAR O PROTÓTIPO

Com base no PRD abaixo, crie um protótipo navegável em HTML, CSS e JavaScript, em um único arquivo, sem bibliotecas externas.

Direção visual: sóbria e institucional, tipografia legível a distância, muito espaço em branco, uma única cor de destaque para a ação principal, sem sombra pesada e sem gradiente decorativo.

Use dados sintéticos realistas e faça os botões navegarem entre as telas. Inclua os quatro estados de cada tela: vazio, carregando, erro e sucesso.

CRITICAR, UMA POR VEZ

A tela [X] está confusa. Quem usa precisa entender em três segundos que [Y]. Reorganize a hierarquia: o mais importante fica maior e vem primeiro. Não mude mais nada.

CONGELAR EM DESIGN.MD

Escreva o DESIGN.md deste protótipo: paleta com os valores, escala tipográfica, escala de espaçamento, estilo de componente e as regras dos quatro estados, de modo que outro desenvolvedor reproduza a interface sem ver o protótipo.



Ambiente

A ferramenta na máquina, o projeto na pasta certa e o banco no ar. Bloco guiado, todos ao mesmo tempo.

Antigravity

Claude Code

Supabase

O Claude Code dentro do Antigravity

A IDE dá a árvore de arquivos, o editor e o terminal. O Claude Code é quem trabalha. Um não substitui o outro.

SEQUÊNCIA DE PREPARAÇÃO, TODOS AO MESMO TEMPO

- 1 Verificar o Node na versão exigida pelo Next.js: `node --version`.
- 2 Instalar: `npm install -g @anthropic-ai/claude-code`.
- 3 Autenticar: rodar `claude` uma vez e concluir o login no navegador.
- 4 Abrir o Antigravity e abrir a pasta do projeto.
- 5 Abrir o terminal integrado e conferir que está dentro da pasta do projeto.
- 6 Rodar `claude` no terminal integrado. A sessão vive na pasta aberta.
- 7 Rodar `/init` para gerar o `CLAUDE.md`.
- 8 Completar o `CLAUDE.md` com as regras do projeto e da instituição.

PROMPT DE COMPLEMENTO DO CLAUDE.MD

Acrescente ao CLAUDE.md: Next.js com App Router e TypeScript, Supabase como banco, Docker para publicar, conforme o SDD; todo texto de interface em português do Brasil; nenhum dado de paciente ou de colaborador, em nenhuma hipótese, apenas dados sintéticos; nenhum segredo, chave ou string de conexão no repositório, sempre por variável de ambiente; antes de qualquer alteração grande, usar o modo plano e me mostrar o plano; ao terminar cada fatia, me dizer exatamente o que testar para saber se funcionou.

Armadilha. Rodar o claude na pasta errada, em geral a pasta pessoal. A sessão passa a enxergar o computador inteiro e perde o foco do projeto.

Os comandos que a turma anota

Seis comandos sustentam a tarde inteira. Os dois últimos são a rede de segurança que permite delegar sem medo.

<code>claude</code>	Abre a sessão do Claude Code na pasta atual
<code>/init</code>	Lê o projeto e escreve o CLAUDE.md
<code>Modo plano</code>	Planeja antes de executar, sem tocar em arquivo
<code>/clear</code>	Limpa o contexto ao trocar de tarefa
<code>git add . && git commit -m "..."</code>	Salva o ponto de retorno
<code>git restore .</code>	Desfaz o que a IA fez desde o último commit

Ganho colateral. O CLAUDE.md deste projeto é o embrião de um padrão institucional da DTIC. Um arquivo desses, mantido pela diretoria e copiado em todo projeto novo, faz a IA respeitar as regras da casa por padrão.

Onde cada coisa mora

A estrutura não é burocracia. É o que permite a IA achar contexto sozinha e outra pessoa assumir o projeto depois.

meu-projeto/

CLAUDE.md	regras do projeto para a IA
README.md	o que é, como roda, onde está publicado
.gitignore	o que nunca entra no repositório
.env.example	os nomes das variáveis, sem os valores
.env.local	os valores reais, fora do repositório

docs/

PRD.md	o que construir
SDD.md	como construir
DESIGN.md	como deve parecer
prototipo.html	a referência visual

app/

components/

lib/

supabase/schema.sql	rotas e telas do Next.js
supabase/seed.sql	componentes de interface

docker/

Dockerfile e compose.yml	cliente Supabase e utilitários
--------------------------	--------------------------------

AS QUATRO REGRAS DA ESTRUTURA

- 1 Documento antes de código. A pasta `docs/` nasce antes da pasta `app/`.
- 2 Segredo nunca no repositório. `.env.example` versionado, `.env.local` ignorado.
- 3 Um lugar por assunto. Banco em `supabase/`, publicação em `docker/`.
- 4 O `README.md` é escrito para quem chega depois, inclusive você daqui a seis meses.

Armadilha. Deixar o PRD e o SDD fora do repositório, no chat ou no computador. Fora do repositório, a IA não lê e o projeto perde a memória.

Postgres gerenciado, sem servidor para cuidar

O Supabase entrega banco, API pronta e controle de acesso por linha. Para um MVP, resolve a camada de dados inteira em minutos.

SEQUÊNCIA

- 1 Criar o projeto, escolher a região mais próxima e guardar a senha do banco.
- 2 Copiar as duas chaves: a pública, prefixada `NEXT_PUBLIC_`, vive no navegador; a de serviço só existe no servidor.
- 3 Abrir o editor de SQL e aplicar o schema que já está no SDD.
- 4 Ligar RLS em todas as tabelas e escrever as políticas de acesso.
- 5 Rodar o seed com dados sintéticos.
- 6 Guardar as variáveis no `.env.local` e registrar os nomes no `.env.example`.

PROMPT DO SCHEMA, A PARTIR DO SDD

Leia o modelo de dados do SDD abaixo e gere dois arquivos SQL para Postgres no Supabase.

schema.sql: tabelas com tipos adequados, chaves primárias, chaves estrangeiras com comportamento de exclusão explícito, restrições de integridade, valores padrão, colunas de criação e atualização, e índices para as consultas do SDD. Em seguida habilite RLS em todas as tabelas e escreva as políticas conforme as regras de acesso.

seed.sql: de trinta a cinquenta linhas de dados sintéticos plausíveis em português do Brasil, cobrindo os casos normais e pelo menos três casos limite.

Comente cada bloco em uma linha explicando a decisão.

As duas chaves e o RLS

Este é o slide que evita o erro mais caro da tarde. Tabela sem política é tabela aberta ao mundo.

Chave pública

Vai para o navegador por definição. Tudo que ela alcança, qualquer visitante alcança. Só é segura porque o RLS limita o que ela vê.

RLS ligado desde o início

Habilitar em todas as tabelas antes de escrever a primeira consulta, e escrever a política junto.

Segredo por variável

Nunca no código, nunca no commit, nunca dentro da imagem Docker.

Chave de serviço

Ignora o RLS. Só existe em código de servidor, nunca prefixada com NEXT_PUBLIC_, nunca no repositório.

Dados sintéticos

Nenhum dado de paciente ou de colaborador entra no banco, nem para testar.

Backup e retenção

Decisão de produção. Fica registrada no SDD mesmo quando adiada.

Armadilha clássica. Desligar o RLS para destravar o desenvolvimento e esquecer de ligar antes de publicar. Conferir na aba de rede do navegador antes do deploy.

IV

Construção e publicação

A IA escreve o código, fatia a fatia. Você verifica, versiona, empacota e publica.

Fatias curtas

Git

Docker

URL

Construir a partir do PRD e do SDD

O prompt de construção não descreve funcionalidade. Ele aponta para os documentos e limita o escopo da vez.

ABERTURA, SEM ESCREVER CÓDIGO

Leia docs/PRD.md, docs/SDD.md e docs/DESIGN.md. Eles são a fonte da verdade deste projeto.

Antes de escrever qualquer código, use o modo plano e me apresente: o que você entendeu do escopo, a ordem das fatias e as decisões que você tomaria e que não estão nos documentos. Não implemente nada ainda.

CADA FATIA

Implemente apenas a fatia [N] do plano de entrega do SDD: [nome da fatia].

Siga o DESIGN.md para a interface e o schema do Supabase para os dados. Não implemente nada de outra fatia nem funcionalidade fora do PRD.

Ao terminar, me diga exatamente o que abrir e o que clicar para verificar, e o que eu devo ver se estiver correto.

QUANDO NÃO FUNCIONA

Quando eu faço [ação], acontece [comportamento observado] e eu esperava [comportamento correto].

Investigue a causa antes de propor solução. Me diga qual é a causa e só então corrija. Não altere nada além do necessário.

O CICLO, REPETIDO FATIA A FATIA

- 1 Pedir uma fatia só.
- 2 Ver funcionando no navegador.
- 3 Corrigir apontando comportamento, não linha de código.
- 4 Commitar assim que funcionar.

Ao final das fatias. "Revise a aplicação inteira contra o PRD e o SDD. Liste em três grupos: implementado e correto, implementado mas divergente, e o que falta. Não corrija nada ainda."

O repositório é a memória do projeto

Git é a rede de segurança que permite delegar sem medo. Se a IA quebrar tudo, um comando volta ao último ponto bom.

<code>git init</code>	Inicia o versionamento, antes de qualquer construção
<code>git status</code>	Mostra o que mudou desde o último commit
<code>git add . && git commit -m "..."</code>	Salva o ponto de retorno ao final de cada fatia
<code>git restore .</code>	Desfaz o que ainda não foi commitado
<code>git log --oneline</code>	O histórico do projeto, uma linha por passo
<code>git checkout -b nome</code>	Abre um ramo para uma mudança arriscada
<code>git push</code>	Envia para o repositório privado da organização

O que nunca entra no repositório. `.env.local`, chaves, senhas, tokens, dados reais de pessoas, dumps de banco e endereços internos de servidor. **Sinal de sucesso:** o histórico conta a construção. Quem ler os commits entende como a aplicação foi feita.

A mesma aplicação em qualquer máquina

O container empacota código, dependências e ambiente. O que roda na sua máquina roda igual no servidor. É isso que o Docker resolve.

OS QUATRO ARQUIVOS

- 1 `Dockerfile`: a receita da imagem, em estágios: dependências, build e execução.
- 2 `.dockerignore`: o que não entra na imagem, incluindo `.env.local` e `.git`.
- 3 `compose.yml`: portas, variáveis, reinício automático, verificação de saúde e rótulos do Traefik.
- 4 `.env` no servidor: os valores reais, fora do repositório.

O CAMINHO ATÉ O AR

- 1 Construir e rodar localmente: `docker compose up --build`.
- 2 Testar em container na própria máquina, com as variáveis do Supabase.
- 3 Enviar o código para o GitHub.
- 4 No servidor da DTIC, clonar em `/opt/nome-do-projeto`.
- 5 Criar o `.env` do servidor com os valores reais.
- 6 Subir: `docker compose up -d --build` e conferir `docker compose logs -f`.

Armadilha. Colocar variáveis de ambiente dentro da imagem. Elas ficam gravadas ali e vazam com a imagem. Variável entra em tempo de execução, sempre.

O prompt do Docker

A estratégia de deploy já está no SDD. O prompt apenas materializa os três arquivos e a regra de atualização.

PROMPT DO EMPACOTAMENTO

Gere o Dockerfile, o .dockerignore e o compose.yml para esta aplicação Next.js, conforme a estratégia de deploy do SDD.

Dockerfile em múltiplos estágios (dependências, build, execução), usando a saída standalone do Next.js, imagem final enxuta, usuário sem privilégio de administrador e porta exposta explicitamente.

No compose.yml: reinício automático, variáveis lidas de arquivo .env, verificação de saúde e rótulos para o Traefik com HTTPS automático no domínio [domínio].

Nenhum segredo escrito nos arquivos. Explique em comentário de uma linha o motivo de cada decisão.

O QUE FICA VALENDO DEPOIS

Atualizar depois

`git pull && docker compose up -d --build`. Esse é o comando que a equipe repete a cada versão.

Sinal de sucesso

A aplicação sobe em container na sua máquina, com as mesmas variáveis, antes de ir ao servidor.

Armadilha

Publicar a partir da pasta local em vez do repositório. O servidor deve clonar, nunca receber cópia manual.

A URL é a entrega

O projeto só existe quando outra pessoa consegue abrir, em outro computador, sem você por perto.

A LISTA DE VERIFICAÇÃO DA PUBLICAÇÃO

- 1 O domínio abre com HTTPS válido.
- 2 A jornada principal funciona de ponta a ponta em outro computador.
- 3 Funciona no celular, em tela de 375 pixels.
- 4 Os quatro estados aparecem: vazio, carregando, erro e sucesso.
- 5 Nenhuma chave de serviço aparece no navegador.
- 6 O RLS está ligado e as políticas fazem o que devem fazer.
- 7 O README diz o que é, para quem, onde está e o que ainda não faz.
- 8 O repositório tem o histórico da construção.

PROMPT DE FECHAMENTO

Escreva o README.md do projeto: o que a aplicação faz e para quem, como rodar localmente, as variáveis de ambiente, como aplicar o schema no Supabase, como publicar e atualizar com Docker, onde está publicada, quais dados guarda e onde, as limitações conhecidas, e a seção Versão 2 do PRD como próximos passos.

A ficha de sustentação, três campos. Dono técnico e dono da dor. Situação: experimento, piloto com um setor ou candidato ao backlog formal. Próxima decisão e data. É o que impede a tarde de virar shadow IT.

Do MVP ao sistema sustentado

O que a tarde deixa de fora de propósito, e a ordem de resolver depois. Ninguém confunde oficina com arquitetura.

1 Autenticação Integração com o diretório institucional e perfis de acesso. PRIMEIRO	2 Persistência Backup, retenção e ciclo de vida do dado definidos. SEGUNDO	3 LGPD Base legal, minimização e registro de tratamento. TERCEIRO	4 Integração Leitura dos sistemas existentes, negociada com quem os sustenta. QUARTO	5 Sustentação Dono formal, monitoramento e plano de descontinuação. QUINTO
--	--	---	--	--

A mensagem. A IA encurtou a construção, não eliminou a engenharia. O MVP publicado é a evidência barata de que vale a pena engenheirar de verdade.

Levar o método, não o projeto

O projeto de hoje resolve uma dor. O método resolve as próximas.

- 1 **A dor em uma frase**, confirmada por quem sofre.
- 2 **PRD podado e SDD** com plano de fatias.
- 3 **Protótipo validado** antes do código.
- 4 **Fatias curtas**, verificadas no navegador, com commit em cada uma.
- 5 **Publicar**, entregar a alguém e definir a próxima decisão.

Ninguém aqui aprendeu a programar hoje. Todo mundo aprendeu a entregar.